

Reflection and Traceability Report on Software Engineering

Team 13, Speech Buddies
Mazen Youssef
Rawan Mahdi
Luna Aljammal
Kelvin Yu

[Reflection is an important component of getting the full benefits from a learning experience. Besides the intrinsic benefits of reflection, this document will be used to help the TAs grade how well your team responded to feedback. Therefore, traceability between Revision 0 and Revision 1 is an important part of the reflection exercise. In addition, several CEAB (Canadian Engineering Accreditation Board) Learning Outcomes (LOs) will be assessed based on your reflections. —TPLT]

1 Changes in Response to Feedback

[Summarize the changes made over the course of the project in response to feedback from TAs, the instructor, teammates, other teams, the project supervisor (if present), and from user testers. —TPLT]

[For those teams with an external supervisor, please highlight how the feedback from the supervisor shaped your project. In particular, you should highlight the supervisor's response to your Rev 0 demonstration to them. —TPLT]

[Version control can make the summary relatively easy, if you used issues and meaningful commits. If your feedback is in an issue, and you responded in the issue tracker, you can point to the issue as part of explaining your changes. If addressing the issue required changes to code or documentation, you can point to the specific commit that made the changes. Although the links are helpful for the details, you should include a label for each item of feedback so that the reader has an idea of what each item is about without the need to click on everything to find out. —TPLT]

[If you were not organized with your commits, traceability between feedback and commits will not be feasible to capture after the fact. You will instead need to spend time writing down a summary of the changes made in response to each item of feedback. —TPLT]

[You should address EVERY item of feedback. A table or itemized list is

recommended. You should record every item of feedback, along with the source of that feedback and the change you made in response to that feedback. The response can be a change to your documentation, code, or development process. The response can also be the reason why no changes were made in response to the feedback. To make this information manageable, you will record the feedback and response separately for each deliverable in the sections that follow. —TPLT]

[If the feedback is general or incomplete, the TA (or instructor) will not be able to grade your response to feedback. In that case your grade on this document, and likely the Revision 1 versions of the other documents will be low. —TPLT]

1.1 SRS and Hazard Analysis

Over the course of the project, we made targeted updates to the SRS and Hazard Analysis in response to TA feedback, advisor guidance, and internal review. The main themes were: (i) improving requirement precision and testability, (ii) clarifying system boundaries and privacy assumptions for a local-first assistive tool, and (iii) strengthening safety language around command execution and user confirmation.

- **Clarified system boundary and privacy assumptions**

Source: TA/advisor feedback during Revision 0 review and demo discussions.

Feedback: The boundary between what runs locally vs. what relies on external components was not explicit enough, which made feasibility and privacy risk harder to assess.

Response: We revised the SRS to state the local-first intent more clearly, identify external dependencies where applicable, and tighten definitions around what user data exists, where it can be stored, and what is not retained by default.

- **Improved requirement specificity and testability**

Source: TA feedback on requirement quality.

Feedback: Some requirements were too high-level or combined multiple behaviours, reducing testability and traceability.

Response: We reworded several requirements to be more measurable (clear triggers, observable outputs, and success criteria) and split bundled requirements so each one maps cleanly to design elements and V&V tests.

- **Strengthened traceability between SRS/HA and the design docs**

Source: TA feedback emphasizing traceability, including the design documentation review ([TA Review #235](#)).

Feedback: The design documentation needed clearer, more consistent module responsibilities and mappings, and the project artifacts needed to remain traceable across documents.

Response: While #235 primarily drove updates to the design documents, addressing it required us to cross-check the design against the SRS and Hazard Analysis. We verified and tightened SRS-to-module and hazard/mitigation-to-module traceability to reduce drift and ensure that safety and privacy assumptions in the SRS/HA were reflected in the architecture.

- **Expanded hazards related to unintended command execution**

Source: TA feedback and internal risk review after implementing command execution flows.

Feedback: The hazard analysis under-emphasized risks from misrecognition, ambiguous intent, and executing harmful browser actions.

Response: We strengthened hazards and mitigations around incorrect transcriptions and ambiguous commands leading to unintended actions. Mitigations were updated to emphasize explicit confirmation for higher-risk actions, clear user feedback during execution, and safe fallback behaviour when commands are out-of-scope or low-confidence.

- **Made privacy and data-retention hazards more concrete**

Source: TA/advisor feedback on privacy risk articulation.

Feedback: Privacy hazards were present but too abstract, making it unclear what data might be logged or retained and what controls exist.

Response: We clarified which artifacts (e.g., transcripts, recordings, logs, and user settings) might exist and refined mitigations to emphasize minimizing stored sensitive data, limiting log exposure, and treating stored artifacts as sensitive by default.

- **Addressed accessibility-related failure modes as safety/usability risks**

Source: TA feedback and usability-oriented review.

Feedback: Since the target users include individuals with dysarthric speech, the analysis should consider failure modes such as unclear prompts, missing confirmations, or confusing system state.

Response: We added/refined hazards capturing user confusion and loss of control due to insufficient feedback, and we connected mitigations to clearer state indications and consistent confirm/cancel interactions.

1.2 Design and Design Documentation

Most of our feedback-driven changes were in the design documentation, where we focused on improving clarity, consistency, and alignment with the implemented system.

- **Improved module boundaries and responsibility descriptions**

Source: TA feedback on ambiguity/overlap between components.

Feedback: Some responsibilities were described in overlapping terms (e.g., session flow vs. command interpretation vs. browser execution), making it hard to understand component ownership.

Response: We revised the Module Guide’s module descriptions to better separate responsibilities (UI/feedback vs. transcription vs. command synthesis vs. execution/orchestration vs. storage/security). This made the architecture easier to understand and reduced ambiguity for implementation and testing.

- **Updated documentation to match the final implementation**

Source: TA/advisor feedback plus internal consistency checks.

Feedback: Some design text reflected earlier architecture drafts rather than the final code structure, creating “documentation drift.”

Response: We updated module descriptions and terminology so they accurately reflect the final architecture and how the system is implemented (including details of the audio capture flow and the model tuning workflow). This improved the MG’s usefulness as a maintenance reference.

- **Strengthened traceability matrices and requirement coverage**

Source: TA feedback emphasizing traceability and completeness.

Feedback: The design needed clearer evidence that requirements (functional, usability, performance, security, etc.) were mapped to concrete parts of the system.

Response: We revised and validated the traceability matrices so each requirement category maps to the relevant modules, and we checked for gaps/duplicates. This also helped ensure the V&V work could be planned directly against the design.

- **Clarified inter-module communication and error handling**

Source: Advisor/TA feedback requesting clearer interaction details.

Feedback: The documents needed clearer description of how modules exchange information and how errors propagate to user-facing feedback.

Response: We expanded the design documentation to describe the message/data flow at a high level (what data types are passed, where session state is tracked, and how failures are turned into user-facing feedback and logs). This made the design more actionable and easier to verify.

- **Refined UI and accessibility documentation based on feedback**

Source: TA feedback and informal user-focused review.

Feedback: The UI story needed clearer user-state feedback (listening, processing, confirm/cancel) and accessibility justification.

Response: We improved UI-related sections and figures/descriptions to emphasize predictable state transitions, clear confirmations, and accessible feedback. These changes were motivated by the needs of users with dysarthric speech, where system confidence and clarity directly affect usability.

1.3 VnV Plan and Report

2 Extras

[Summarize the extras that were tackled by this project. Extras can include usability testing, code walkthroughs, user documentation, formal proof, Gender-Mag personas, Design Thinking, etc. Extras should have already been approved by the course instructor as included in your problem statement. —TPLT]

3 Design Iteration (LO11 (PrototypeIterate))

[Explain how you arrived at your final design and implementation. How did the design evolve from the first version to the final version? —TPLT]

[Don't just say what you changed, say why you changed it. The needs of the client should be part of the explanation. For example, if you made changes in response to usability testing, explain what the testing found and what changes it led to. —TPLT]

3.1 Shortcut Manager Iteration

One important design iteration in VoiceBridge was the development of the **ShortcutManager**. In the earlier versions of the system, commands were handled one at a time with no persistent way to record, save, and replay common action sequences. While this was enough to demonstrate basic browser control, it did not match the client need for efficiency and accessibility in repeated everyday tasks. For a user with impaired speech, repeatedly issuing the same sequence of voice commands can increase fatigue and reduce the overall usability of the system. Because of this, we introduced shortcuts as a way to reduce the number of spoken interactions required for frequent tasks.

Our first idea was relatively simple: store a shortcut as a sequence of recorded commands and replay them later. However, once we integrated this into the live system, we found that shortcut behavior could conflict with the runtime state machine. For example, the user could try to begin a shortcut recording while the system was still processing another command, or multiple parts of the backend could attempt to modify shortcut state at the same time. This was especially important because VoiceBridge is a real-time system with asynchronous events, including speech capture, transcription, intent resolution, and browser execution. As a result, we revised the design so that shortcut recording became an explicitly managed subsystem rather than just an added list of commands.

The final design introduced a dedicated **ShortcutManager** module with a clear internal state, persistent storage, and locking for safe concurrent access. Instead of scattering shortcut logic across the codebase, the manager became responsible for starting and stopping recordings, storing recorded commands, validating shortcut names, and loading/saving shortcuts from disk. This design was chosen to improve maintainability and prevent race conditions in cases where multiple backend events happen close together. The use of a lock was an important iteration because testing showed that shortcut actions could overlap with other command-processing flows in unpredictable ways if the state was not protected.

We also changed how shortcuts were represented. In earlier thinking, shortcuts were treated mainly as temporary in-memory recordings for the active session. This was limiting, because a user would lose their shortcuts between runs of the application. Since one of the client goals was to make the system practical for daily use, we moved to persistent storage using a local JSON-based approach. This allowed shortcuts to survive application restarts and made the feature more realistic as an assistive tool rather than just a demo feature. The trade-off was that we now needed validation and structured file handling, but this was worth it because it supported a more reliable user experience.

Another design change was motivated by usability. Initially, shortcut support was focused only on storing the raw sequence of actions. As the design matured, we recognized that the feature also needed clear start/stop control and user feedback so that recording would feel predictable. Without this, a user could be unsure whether the system was actively recording commands into a shortcut or just executing them normally. This was especially important given our client focus on accessibility and confidence in system behavior. We therefore refined the design so that shortcut recording had explicit transitions, clear prompts, and tighter integration with the system state flow.

Overall, the **ShortcutManager** evolved from a simple convenience idea into a more structured subsystem designed around accessibility, predictability, and safe execution. The main reason for these changes was not just technical cleanup, but the need to better support users who benefit from reducing repeated speech effort while still being able to trust what the system is doing.

4 Design Decisions (LO12)

[Reflect and justify your design decisions. How did limitations, assumptions, and constraints influence your decisions? Discuss each of these separately. —TPLT]

5 Economic Considerations (LO23)

[Is there a market for your product? What would be involved in marketing your product? What is your estimate of the cost to produce a version that you could sell? What would you charge for your product? How many units would

you have to sell to make money? If your product isn't something that would be sold, like an open source project, how would you go about attracting users? How many potential users currently exist? —TPLT]

6 Reflection on Project Management (LO24)

6.1 How Does Your Project Management Compare to Your Development Plan

Our project management aligned well with our Development Plan. We maintained a consistent meeting rhythm to coordinate priorities and remove blockers, and we used asynchronous communication between meetings to keep work moving. We used GitHub Issues and pull requests to plan tasks, record feedback, and track progress across deliverables. Team roles were clear and helped us work efficiently, while still allowing collaboration across areas when needed. We also used the technologies we planned on using (GitHub for version control and coordination, and LaTeX for documentation), and we kept our workflow structured around reviewable changes and traceability.

6.2 What Went Well?

- **Issue tracking and traceability:** Capturing peer and TA feedback as GitHub Issues made it easy to plan changes and clearly demonstrate traceability from comments to updates.
- **Clear ownership with collaboration:** Assigning primary owners for major deliverables improved consistency and helped each document progress steadily, while collaboration ensured quality and shared understanding.
- **Review and iteration:** Regular internal review cycles improved clarity, consistency, and alignment across the SRS/HA, design documentation, and V&V artifacts.
- **Coordination across implementation and documentation:** Keeping documentation connected to the implementation helped ensure design decisions were well-justified and supported by the final system.

6.3 What Went Wrong?

- **Continuous alignment effort:** Maintaining alignment between requirements, design, and V&V artifacts required ongoing attention, and the experience reinforced the value of regularly revisiting traceability links as the system evolves.
- **Nonfunctional evaluation planning:** Defining measurable nonfunctional tests and presenting quantitative results clearly was a valuable learning area that strengthened the quality of our V&V work.

- **Estimating integration work:** Coordinating integration across components highlighted the importance of explicitly planning interface details and test hooks early, which improved the final system robustness.

6.4 What Would you Do Differently Next Time?

- **Make traceability updates routine:** Update traceability matrices on a regular schedule so cross-document consistency is maintained continuously.
- **Define acceptance criteria for issues:** Add short, explicit acceptance criteria to issues to keep scope clear and improve verification.
- **Plan earlier nonfunctional baselines:** Establish performance and usability baselines earlier so evaluation is repeatable and comparisons across iterations are straightforward.

7 Ethics and Equity (LO18)

Equity and universal design were central to VoiceBridge because the system is intended to support individuals with dysarthric speech. We designed for clarity, predictability, and user control by emphasizing clear system-state feedback (listening, processing, executing), confirmation for higher-risk actions, and safe handling of low-confidence or out-of-scope commands. We also treated privacy as part of equitable design by documenting data assumptions and encouraging local-first handling where feasible, so users can understand and trust how their information is processed. Across the project, we aimed to reduce unnecessary cognitive load and promote confidence by keeping interactions consistent and transparent.

8 Reflection on Capstone

8.1 Which Courses Were Relevant

- **SFWRENG 4HC3 (Human-Computer Interaction):** Informed our focus on usable, predictable interactions, user feedback, and accessibility-oriented design decisions.
- **SFWRENG 3A04 (Software Architecture):** Supported our approach to module decomposition, clear responsibility assignment, and documenting interfaces and system structure.
- **SFWRENG 4AL3 (Applied Machine Learning):** Helped us reason about ASR-related considerations such as evaluation, error modes, and performance tradeoffs in practical settings.

8.2 Knowledge/Skills Outside of Courses

- **End-to-end ASR pipeline integration:** Building and integrating a real-time flow from audio capture to transcription to command execution.
- **Practical evaluation:** Designing repeatable procedures and metrics (e.g., latency and error characterization) that reflect real usage.
- **Privacy-aware engineering:** Translating privacy expectations into concrete requirements, hazards, and mitigations for an assistive tool.
- **Documentation and maintenance discipline:** Keeping requirements, design, and V&V artifacts consistent through traceability as the project evolved.
- **Collaborative workflow:** Applying issue tracking, pull requests, and structured reviews to coordinate work and justify changes.